

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2,BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks: 50

Annexure A

Case Study

Duration: 2hrs

1. A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays. Analyze how the components of a Markov Decision Process (states, actions, rewards, policy, and environment) can be defined for this system. Design a reward function that balances speed, accuracy, and safety, and evaluate how reward design influences the robot's learning and behavior. **(5 Marks)**
2. A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history. Apply RL concepts to construct a suitable state representation, action space, and reward function. Analyze how different reward signals affect user engagement and evaluate the trade-off between exploration and exploitation. **(5 Marks)**

3. An autonomous agricultural system uses RL to optimize irrigation and fertilizer usage based on soil conditions and weather forecasts. Design an RL framework by defining states, actions, and rewards. Analyze the effect of delayed rewards on crop yield optimization and justify the choice of a suitable learning algorithm. **(5 Marks)**
4. A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making. **(5 Marks)**
5. A ride-sharing company plans to use RL to optimize driver allocation and dynamic pricing strategies. Apply RL concepts to define states, actions, and rewards. Analyze how environmental uncertainty affects learning and evaluate ethical concerns related to pricing strategies. **(5 Marks)**
6. A smart home automation system uses RL to control lighting, heating, and cooling based on occupancy and weather conditions. Design an RL framework including states, actions, and reward function. Analyze how conflicting objectives like comfort and energy saving can be handled and evaluate the impact of reward shaping. **(5 Marks)**
7. A gaming company is developing an RL-based agent to play a complex strategy game against human players. Analyze how the problem can be modeled as an MDP. Design a reward structure that encourages long-term strategy and evaluate how exploration strategies affect performance. **(5 Marks)**
8. A financial institution uses RL to detect fraudulent transactions in real time. Apply RL principles to define the state space, action space, and reward function. Analyze the challenges of imbalanced data and delayed feedback, and evaluate the risks associated with incorrect decisions. **(5 Marks)**
9. A healthcare system uses RL to recommend personalized rehabilitation exercises for patients based on their progress. Design an RL framework with suitable states, actions, and rewards. Analyze the impact of delayed rewards and patient variability, and evaluate ethical concerns and safety considerations. **(5 Marks)**

10. A smart grid system uses RL to manage electricity distribution and reduce peak load demand. Analyze how the components of an MDP can be defined in this context. Design a reward function to balance efficiency and reliability, and evaluate the challenges of scalability and real-time learning. **(5 Marks)**

Code of Conduct:

*I **BANDLAPALLI BHANUTEJA REDDY(192325016)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. BHANUTEJA REDDY

RUBRICS FOR CASE STUDY

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured , and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWER SHEET

MLA03 – Reinforcement Learning | CO1 – Assessment Tool 2 – Case Study (Detailed Answers per Rubric)

Answer 1

Understanding of Case (25%)

- The core problem is a warehouse robot that must complete a multi-stage task — picking, packing, and transporting items — under two simultaneous constraints: avoiding obstacles (a safety/hard constraint) and minimizing delays (an efficiency/soft objective).
- Because these two goals can conflict (moving fast increases collision risk, moving cautiously increases delay), the case requires a reward function that is not just about task completion but about balancing competing priorities — this is the key issue the RL design must resolve.

Application of Concepts (25%)

- **State:** the robot's (x, y) position and orientation, the location/status of the item being handled (unpicked, picked, packed, in-transit), a local obstacle/occupancy map, and the current stage of the task (pick / pack / transport).
- **Action:** discrete movement actions (forward, turn left/right, reverse), plus task-specific actions such as pick-item, pack-item, and release-item.
- **Environment:** the warehouse layout — shelving units, aisles, charging stations, and dynamic obstacles such as other robots or staff — modelled as a grid or continuous 2D space with occupancy information.
- **Policy:** a function $\pi(\text{state}) \rightarrow \text{action}$, trained so that, in any given state, it selects the action that is expected to lead to the fastest, safest completion of the current sub-task.
- **Reward function:** $R = w_1 \cdot (\text{speed_bonus}) + w_2 \cdot (\text{accuracy_bonus}) - w_3 \cdot (\text{collision_penalty}) - w_4 \cdot (\text{time_penalty})$, where speed_bonus rewards reaching milestones quickly, accuracy_bonus rewards correct item identification/placement, and the penalty terms discourage collisions and idling.

Analysis (20%)

- If w_1 (speed) dominates the reward, the learned policy tends to cut corners near shelving and other robots, increasing real-world collision risk even if the simulator shows high reward — a classic case of reward hacking driven by poor weighting.
- If w_3 (safety) dominates too strongly, the robot may adopt an overly conservative policy that stops or reroutes at the slightest obstacle proximity, inflating delivery time and reducing warehouse throughput — so safety weighting has diminishing and eventually counter-productive returns for the business objective.

- Accuracy needs its own explicit reward term rather than being folded into speed, because a fast-but-wrong pick (wrong SKU, damaged item) creates costly downstream correction work that a pure speed/safety trade-off would not capture.

Solution Design (20%)

- A practical design fixes safety as a near-hard constraint (a large, fixed negative penalty for any collision or near-miss, e.g. -50, independent of scale) so it is rarely worth trading off, while speed and accuracy are tuned as softer, scaled rewards (e.g. +1 per second saved, +10 for a correct pick) that the policy can trade off against each other more flexibly.
- Reward shaping can also be introduced gradually during training — starting with a heavier safety penalty early in training to establish safe habits, then gradually increasing the relative weight on speed once the policy reliably avoids collisions, a curriculum-style approach to reward design.

Clarity (10%)

- All five MDP components are explicitly separated and defined with concrete detail, the reward formula is stated numerically, and the trade-off reasoning follows a clear cause-and-effect structure from weighting choice to robot behaviour.

Answer 2

Understanding of Case (25%)

- The case is a movie/show recommendation engine that must personalize suggestions dynamically as user preferences evolve, rather than relying on a static, one-time model of taste — this framing naturally suits RL because each recommendation is a sequential decision that affects future user behaviour.

Application of Concepts (25%)

- **State:** a representation of the user's recent viewing history (genres watched, ratings given, completion rates), session context (time of day, device type), and possibly an embedding of long-term preference derived from past interactions.
- **Action:** selecting one title (or a ranked list of titles) from the catalog to present to the user at a given moment.
- **Reward:** a composite signal — +1 for a click, +5 for watching to completion, -2 for skipping within the first few minutes, and a smaller positive trickle reward for re-engagement (returning to watch more later), so the reward reflects genuine satisfaction rather than just initial interest.

Analysis (20%)

- Optimizing purely for clicks tends to surface sensational thumbnails/titles ('clickbait') that generate short-term engagement metrics but erode long-term trust and satisfaction once users repeatedly feel misled by the recommendation.
- Optimizing for completion or watch-time better proxies genuine enjoyment but can bias recommendations toward already-popular, safe content, reducing catalog diversity and discovery of niche titles the user might actually prefer.
- The exploration/exploitation trade-off is central here: pure exploitation (always recommending what's statistically closest to past behaviour) maximizes short-term reward but risks a narrowing 'filter bubble', while too much exploration (frequently trying unfamiliar genres) risks user frustration and abandonment of the platform.

Solution Design (20%)

- An ϵ -greedy or Upper Confidence Bound (UCB) strategy is well suited: the system exploits known preferences most of the time (say, 90% of recommendation slots) while reserving a smaller fraction for exploratory picks in adjacent genres, allowing the model to keep discovering evolving tastes without materially harming session satisfaction.
- A multi-objective reward (blending short-term engagement with a longer-horizon retention signal, e.g. did the user return within a week) further discourages the system from over-optimizing for one metric at the expense of sustained platform loyalty.

Clarity (10%)

- Each RL component maps directly and specifically to the recommendation scenario, and the reasoning about reward-signal choice explicitly connects each option (clicks vs. completion vs. retention) to a concrete behavioural consequence.

Answer 3

Understanding of Case (25%)

- The case is an autonomous agricultural system that must decide irrigation and fertilizer application using soil and weather data, where the true measure of success — crop yield — is only observed at harvest, long after most of the relevant actions were taken.

Application of Concepts (25%)

- **State:** current soil moisture and nutrient levels, recent weather history, upcoming forecast, and the crop's current growth stage.
- **Action:** irrigate by a chosen volume, apply a specific type/amount of fertilizer, or take no action for the current time step.
- **Reward:** intermediate proxy signals such as soil-health indicators and plant growth-rate measurements during the season, plus the dominant, delayed final reward tied to the harvested yield and its quality grade.

Analysis (20%)

- Because the season-end yield is the only 'ground truth' reward and it arrives weeks or months after the actions that caused it, the agent faces a severe temporal credit-assignment problem: it must determine which of many irrigation/fertilizer decisions across the season actually contributed to (or hurt) the final yield.
- Delayed rewards also make the learning signal noisy, since external factors outside the agent's control (unexpected weather events, pests) can affect yield independent of the agent's decisions, diluting the direct causal link between action and outcome.

Solution Design (20%)

- A discount factor γ close to 1 is justified so that early-season actions are not unfairly discounted relative to the final yield reward, allowing the value function to properly attribute long-term consequences to early irrigation/fertilizer choices.
- Eligibility traces (as in $TD(\lambda)$) or a Deep Q-Network with reward shaping using intermediate soil/growth indicators can bridge the gap between sparse final rewards and the need for more frequent learning signals, effectively turning a single delayed reward into a smoother trajectory of partial credit.

Clarity (10%)

- The link between delayed rewards, the credit-assignment problem, and the specific algorithmic remedy (high γ , eligibility traces, intermediate shaping) is made explicit and logically sequenced.

Answer 4

Understanding of Case (25%)

- The case is a real-time network intrusion detection and response system, which differs from typical classification tasks because the environment is adversarial (attackers adapt) and decisions must be made and acted upon immediately, not just predicted.

Application of Concepts (25%)

- **State:** network traffic features such as packet rate, protocol type, source/destination address patterns, and deviation from a baseline 'normal' traffic profile.
- **Environment:** the live, non-stationary network, where the 'opponent' (an attacker) actively changes tactics over time, unlike a static classification dataset.
- **Action:** allow the traffic, flag it for human/analyst review, temporarily block the source, or trigger an automated alert/escalation.
- **Reward:** +10 for correctly identifying and stopping an actual attack, -20 for a missed attack (false negative, since the cost of an undetected breach is severe), and -5 for a false positive that disrupts legitimate business traffic.

Analysis (20%)

- Attacks are a small fraction of all network events, so the reward signal for correct attack detection is sparse — the agent experiences long stretches of 'normal' states with little informative feedback, risking a policy that trivially learns to always predict 'normal' since it is right most of the time by volume.
- Real-time decision-making adds a further constraint: the agent cannot afford lengthy deliberation or extensive simulated look-ahead per packet, since detection and response must happen within milliseconds to be useful, unlike offline RL training which can be slow.

Solution Design (20%)

- Prioritized experience replay, which over-samples the rare attack events relative to their true frequency during training, directly counteracts the sparse-reward/imbalance problem without requiring more real attack data than is naturally available.
- Deploying a lightweight, pre-trained policy network for real-time inference (with heavier retraining done offline/periodically on accumulated logs) resolves the real-time constraint, since the expensive learning happens asynchronously while the live decision step remains a fast forward pass.

Clarity (10%)

- The adversarial and non-stationary nature of the environment is explicitly distinguished from a standard static classification setting, and each named challenge (sparsity, real-time constraint) is paired with a matching, specific mitigation.

Answer 5

Understanding of Case (25%)

- The case is a ride-sharing platform that must jointly solve two coupled problems — where to allocate drivers and how to price rides dynamically — under uncertain and rapidly changing demand, while also facing legitimate ethical scrutiny over pricing fairness.

Application of Concepts (25%)

- **State:** current driver locations and availability, incoming ride requests, time-of-day/day-of-week patterns, weather, and local event data (e.g. concerts, sports games) that can spike demand.
- **Action:** match a specific driver to a request, reposition idle drivers toward anticipated demand zones, or set a surge-pricing multiplier for a given area/time window.
- **Reward:** +revenue for each completed ride, adjusted by −penalty for long rider wait times, unmatched requests, or driver idle time, capturing both business performance and service quality.

Analysis (20%)

- Demand uncertainty is substantial and can shift abruptly (sudden weather changes, unplanned events), making the environment partially observable in practice — the agent must generalize across many demand patterns rather than memorizing a fixed daily schedule, and its policy needs to be robust to distribution shift.
- Pure revenue-maximizing pricing can produce ethically troubling outcomes: surge pricing during emergencies (e.g. severe weather, a security incident) can price out riders who most need transportation, raising fairness and public-relations risks that a naive reward-maximizing agent would not account for on its own.

Solution Design (20%)

- A constrained or shaped reward that caps the maximum surge multiplier, or that adds an explicit fairness penalty term during declared emergencies, prevents the policy from exploiting demand spikes in ethically harmful ways while still allowing normal dynamic pricing during ordinary demand fluctuations.
- For the uncertainty challenge, using a model that explicitly represents demand as a probability distribution (rather than a point estimate) — combined with periodic retraining on recent data — helps the policy remain robust as demand patterns evolve over weeks and seasons.

Clarity (10%)

- The technical demand-uncertainty challenge and the ethical pricing-fairness concern are treated as two clearly separated issues, each with its own explanation and its own targeted solution.

Answer 6

Understanding of Case (25%)

- The case is a smart home automation system controlling lighting, heating, and cooling based on occupancy and weather, where the central design challenge is that comfort and energy efficiency are frequently in direct tension with each other.

Application of Concepts (25%)

- **State:** occupancy sensor readings (which rooms are occupied), current indoor temperature per room, outdoor temperature/weather forecast, and time of day.
- **Action:** adjust the thermostat setpoint up/down, switch HVAC modes (heating/cooling/off), and turn lights on/off or dim them to a target level.
- **Reward:** $R = \text{comfort_score} - \lambda \cdot \text{energy_used}$, where *comfort_score* rewards keeping occupied rooms within a target temperature/lighting band and the energy term penalizes total consumption, with λ controlling how aggressively energy savings are pursued relative to comfort.

Analysis (20%)

- Maintaining an ideal, narrow comfort range typically requires more continuous heating/cooling energy, so any single scalar reward necessarily encodes an implicit trade-off — the choice of λ effectively decides the household's comfort-vs-savings philosophy, and different households may want different λ values.
- A poorly shaped reward (e.g. a hard temperature cutoff rather than a gradient) can cause abrupt, energy-wasteful on/off cycling of HVAC systems as the agent chases a rigid threshold rather than settling into an efficient steady state.

Solution Design (20%)

- Reward shaping with a smooth (e.g. quadratic) penalty around the target comfort range — rather than a step-function penalty — allows the agent to learn fine-grained trade-offs, such as slightly relaxing temperature control in unoccupied rooms while maintaining tighter control where people currently are.
- Allowing λ to be a user-configurable parameter (rather than fixed by the system designer) lets each household explicitly express its own comfort/savings preference, turning an implicit conflict into an explicit, tunable design choice.

Clarity (10%)

- The comfort-vs-energy conflict is expressed as a precise mathematical trade-off (the λ -weighted reward), and the reward-shaping recommendation is directly tied to avoiding a specific, well-explained failure mode (on/off cycling).

Answer 7

Understanding of Case (25%)

- The case is an RL agent for a complex strategy game against human players, which must be modelled as a long-horizon sequential decision problem where the only unambiguous reward (win or lose) arrives only at the very end of a potentially long match.

Application of Concepts (25%)

- **State:** the current game board/resource configuration at a given turn, including unit positions, available resources, and known information about the opponent.
- **Action:** the set of legal moves available at that turn — building units/structures, moving pieces, attacking, or passing.
- **Reward:** a terminal reward of +1 for winning and -1 for losing the match, supplemented with small intermediate shaping rewards (e.g. for territory gained, resources secured, or units preserved) to give the agent more frequent learning signal without distorting the ultimate win/loss objective.

Analysis (20%)

- A pure win/loss reward is extremely sparse over a long game with many decision points, so exploration strategy has an outsized effect on outcomes: too little exploration causes the agent to converge prematurely on a narrow, exploitable strategy, while excessive exploration wastes moves on lines of play unrelated to a coherent long-term plan.
- Because the game is adversarial (a thinking opponent, not a static environment), exploration must also account for opponent adaptation — a fixed strategy discovered through early exploration may become exploitable once human opponents learn to counter it.

Solution Design (20%)

- Monte Carlo Tree Search combined with a learned policy/value network (in the style of AlphaZero) is well suited: the tree search provides principled, look-ahead-based exploitation of promising lines, while random rollouts and self-play against varied opponent strategies provide the exploration needed to avoid a narrow, exploitable policy.
- Maintaining a pool of past policy versions to play against during training (self-play with historical opponents) further guards against overfitting to a single exploitable strategy, keeping the learned policy robust against diverse human play styles.

Clarity (10%)

- The long-horizon, sparse-terminal-reward structure is clearly connected to the exploration-strategy discussion, and the solution (MCTS plus self-play) is explained in terms of the specific problem it solves rather than named generically.

Answer 8

Understanding of Case (25%)

- The case is a real-time financial fraud-detection system, where the central challenges are that fraudulent transactions are rare relative to legitimate ones, and the true label (confirmed fraud or not) is often only available well after the transaction decision must be made.

Application of Concepts (25%)

- **State:** transaction-level features such as amount, merchant category, geographic location, transaction frequency, and deviation from the account's historical spending pattern.
- **Action:** approve the transaction, decline it outright, or flag it for manual analyst review.
- **Reward:** +reward for correctly approving legitimate transactions and correctly blocking fraud, a large -penalty for a missed fraud (false negative), and a smaller -penalty for a false decline (false positive) that inconveniences a genuine customer.

Analysis (20%)

- Imbalanced data (very few confirmed fraud cases relative to legitimate transactions) biases a naively trained policy toward almost always approving, since that maximizes raw accuracy on the majority class while catastrophically failing on the rare, high-cost fraud cases.
- Delayed feedback compounds this: confirmed fraud labels frequently arrive days after the transaction (through customer disputes or investigations), so the agent must act on transaction-level features alone at decision time, without immediate confirmation of whether its past decisions were correct.
- The risk of incorrect decisions is asymmetric and significant either way: a missed fraud causes direct financial loss and potential regulatory exposure, while an incorrect decline damages customer trust and can drive customers to competitors — both risks must be weighed, not just minimized in aggregate.

Solution Design (20%)

- Asymmetric reward penalties (making a missed fraud cost substantially more than a false decline in the reward function) directly encodes the real-world cost asymmetry into the learned policy, rather than relying on a naive accuracy-based objective.
- Oversampling or replaying known fraud cases during training (addressing the imbalance) combined with a short-term proxy reward — such as a rule-based or statistical suspicion score used until the delayed ground-truth label arrives — allows the agent to keep learning promptly rather than waiting on slow-arriving labels.

Clarity (10%)

- Data imbalance and delayed feedback are treated as two distinct, named challenges, each explained mechanistically, with the decision-risk trade-off made explicit rather than left implicit.

Answer 9

Understanding of Case (25%)

- The case is a healthcare RL system recommending personalized rehabilitation exercises, which is especially sensitive because recovery outcomes are delayed, patients vary enormously in their response to treatment, and mistakes can directly harm patient wellbeing.

Application of Concepts (25%)

- **State:** the patient's current recovery progress (mobility/strength scores), self-reported pain level, and recent exercise-completion history.
- **Action:** recommend a specific exercise type and intensity level, or recommend rest, for the next session.
- **Reward:** +reward for measurable improvement in mobility or strength over successive assessments, and -penalty for reported pain spikes, missed sessions, or setbacks in recovery.

Analysis (20%)

- Recovery outcomes are only observable over days or weeks, so — as in the agricultural case — the system faces a credit-assignment problem in determining which specific exercise recommendations contributed to an observed improvement or setback.
- Patient variability means the same exercise intensity that helps one patient recover faster may cause a setback or injury in another with different baseline fitness, comorbidities, or pain tolerance, so a single 'globally optimal' policy learned by pooling all patients' data risks being unsafe for outlier patients.
- Ethical and safety concerns are paramount here in a way that outweighs pure optimization: an aggressive exploration strategy that tries a new, untested exercise intensity purely to gather more training data could directly injure a vulnerable patient.

Solution Design (20%)

- Constraining the action space to only clinician-pre-approved exercise intensities for each patient (rather than letting the RL agent freely explore the full action space) ensures that exploration never exceeds medically safe bounds.
- Using a risk-averse or conservative exploration strategy — such as only trying a new intensity level after clinician sign-off, or using patient-specific personalization (e.g. contextual bandits with patient features) rather than a single shared policy — respects individual variability while still allowing the system to adapt over time.

Clarity (10%)

- Delayed feedback, patient variability, and safety/ethical risk are each treated as separate, clearly explained concerns, and the solution design explicitly subordinates optimization speed to patient safety.

Answer 10

Understanding of Case (25%)

- The case is a smart grid system managing electricity distribution to reduce peak load, which must operate reliably and efficiently at a large scale spanning many interconnected zones, consumers, and generation sources — introducing both a scalability challenge and a real-time operational constraint.

Application of Concepts (25%)

- **State:** current electricity demand across zones, available generation capacity (including variable renewable sources), time of day, and weather conditions affecting both demand and renewable supply.
- **Action:** redistribute power allocation across zones, activate or deactivate reserve generation capacity, or issue demand-response signals asking consumers to shift usage away from peak periods.
- **Reward:** +reward for meeting demand while keeping voltage/frequency within safe operating limits, -penalty for peak-load overshoot, blackouts, or excessive reliance on costly reserve generation, balancing cost efficiency against reliability.

Analysis (20%)

- As the grid scales to cover more zones, consumers, and generation assets, the state and action spaces grow combinatorially, making a tabular RL approach infeasible and requiring function approximation to generalize across the very large space of possible grid states.
- The grid must also respond to demand fluctuations in near real time (seconds to minutes), so the system cannot afford lengthy re-planning at each decision point the way an offline planning system might, creating tension between decision quality and decision speed.

Solution Design (20%)

- Deep function approximation (e.g. a deep neural network estimating Q-values or a policy network) addresses the scalability challenge by generalizing across similar grid states rather than requiring an explicit value for every possible configuration.
- A hierarchical or zone-decomposed control architecture — where local zone controllers make fast, local decisions and a higher-level coordinator only intervenes for cross-zone balancing — reduces the effective decision space at each level and supports the real-time response requirement without needing a single monolithic real-time optimizer over the entire grid.

Clarity (10%)

- Scalability and real-time responsiveness are identified as two distinct, well-explained challenges, each matched with a specific architectural solution (function approximation, hierarchical control) rather than a generic reference to 'deep learning'.